# IoT Firmware Patch Diffing & CVE Correlation

Automated Binary Analysis Pipeline for OpenWRT Firmware (MIPS/ARM)

Amadou Tidiane Anne
Université de Bretagne Occidentale
Master 1 Embedded Security
Brest, France

## Abstract

Ce rapport documente la conception et l'implémentation d'un outil automatisé de *patch diffing* sur firmwares IoT. L'outil compare deux versions d'un firmware binaire (OpenWRT 22.03.3 → 22.03.5, TP-Link WDR4300 v1, architecture MIPS), identifie les fonctions modifiées via Ghidra scripting, et corrèle ces modifications avec les CVE publiés par la National Vulnerability Database (NVD/NIST). Le pipeline complet — extraction Binwalk, désassemblage Ghidra, moteur de diff multi-critères (hash SHA-256 + similarité Jaccard) et corrélation CVE — est présenté avec ses résultats, ses limites et ses pistes d'amélioration. Le **CVE-2023-24182** (XSS stocké dans LuCI) a été confirmé et analysé ; la corrélation des CVE OpenSSL reste à finaliser suite à un bug de procédure Ghidra identifié et diagnostiqué programmatiquement.

## CCS Concepts

• **Security and privacy** → **Embedded systems security**; *Vulnerability management*; • **Software and its engineering** → *Binary analysis.*

## Keywords

patch diffing, firmware IoT, Ghidra, CVE, MIPS, OpenWRT, reverse engineering, vulnérabilité binaire

**ACM Reference Format:**
Amadou Tidiane Anne. 2026. IoT Firmware Patch Diffing & CVE Correlation: Automated Binary Analysis Pipeline for OpenWRT Firmware (MIPS/ARM). In *Proceedings of Projet Embedded Security — Master 1 (UBO 2026).* ACM, New York, NY, USA, 4 pages.

## 1 Introduction & Contexte

### 1.1 Problématique

Lorsqu'un fabricant IoT publie une mise à jour de firmware, il corrige souvent des vulnérabilités sans documenter précisément les fonctions affectées. Cette opacité entraîne deux difficultés : les administrateurs système ignorent ce qui a réellement été corrigé, et les chercheurs ne peuvent pas vérifier l'efficacité du patch.

Le **patch diffing** répond à cette question en comparant directement les binaires des deux versions. Cette approche est utilisée par Google Project Zero (patches Microsoft/Apple), Quarkslab (firmwares IoT) et le CERT/CC (validation ICS/SCADA).

*UBO 2026, UBO, Brest, France*
2026.

### 1.2 Architecture du pipeline

```
fw_v1.bin → Binwalk → SquashFS → Ghidra → hash_v1.json ↘
fw_v2.bin → Binwalk → SquashFS → Ghidra → hash_v2.json ↗
          → diff_engine.py → modified_functions.json
          → nvd_correlator.py → rapport final
```

## 2 Environnement & Préparation

### 2.1 Stack technique

| Outil | Rôle |
|---|---|
| Binwalk | Extraction et analyse du firmware binaire |
| Ghidra | Désassemblage et analyse statique (NSA) |
| GhidraScript | API Python/Jython – automatisation |
| capstone | Désassembleur Python (analyse opcodes) |
| NVD API | Base de données CVE/NIST |
| Python 3 | Orchestration, diffing, reporting |

**Table 1: Stack technique du projet**

### 2.2 Firmwares cibles

Firmwares **OpenWRT** sélectionnés pour : (1) leur caractère open-source permettant la vérification contre le code source ; (2) leurs CVE documentés permettant la corrélation ; (3) leur architecture MIPS/ARM représentative du parc IoT réel.

| Version | Identifiant firmware |
|---|---|
| 22.03.3 | `tplink_tl-wdr4300-v1-. . . -sysupgrade.bin` |
| 22.03.5 | `tplink_tl-wdr4300-v1-. . . -sysupgrade.bin` |

**Table 2: Firmwares cibles (TP-Link WDR4300 v1)**

## 3 Phase 1 — Extraction des Firmwares

### 3.1 Analyse Binwalk

**Listing 1: Sortie Binwalk – v22.03.3 (extrait)**

```
DECIMAL     HEX        DESCRIPTION
0           0x0        uImage kernel, LZMA, MIPS32
2713605     0x296805   SquashFS v4.0, xz,
                       3899142 bytes
```

**Observation :** Le passage noyau 5.10.161 → 5.10.176 et l'augmentation des inodes (1366 → 1368) confirment l'intégration de correctifs de sécurité entre les deux versions.

| Caractéristique | v22.03.3 | v22.03.5 |
|---|---|---|
| Noyau Linux | 5.10.161 | 5.10.176 |
| Compression | LZMA | LZMA |
| Offset SquashFS | `0x296805` | `0x2989D7` |
| Architecture | MIPS32 Little Endian | |
| Taille SquashFS | 3 899 142 o | 3 904 878 o |
| Inodes | 1 366 | 1 368 |

**Table 3: Métadonnées Binwalk — v33 vs v35**

## 3.2 Identification des binaires modifiés

Un diff SHA-256 sur l'ensemble des deux rootfs (150+ fichiers) identifie les cibles prioritaires :

| Binaire | CVE probable | Modifié |
|---|---|---|
| `libc.so` | CVE-2023-0464/0465 | [OUI] |
| `wpad` | OpenSSL/TLS | [OUI] |
| `libuclient.so` | HTTP/HTTPS client | [OUI] |
| `liblucihttp.so` | Interface LuCI | [OUI] |
| `uhttpd` | — | [NON] |
| `dnsmasq` | — | [NON] |

**Table 4: Diff SHA-256 sur rootfs complet (150+ fichiers)**

**Cible retenue :** `libuclient.so` (SHA-256 distincts)
v33 : `26e11c0a. . . acaa76f3`
v35 : `3d86e323. . . fde03ad8`

# 4 Phase 2 — Analyse Ghidra

## 4.1 GhidraFunctionHasher.py

Plutôt que de comparer les adresses mémoire (variables à chaque compilation), le script extrait les *mnémoniques* de chaque instruction (`sw`, `jalr`, `lw`...) et calcule leur hash SHA-256 concaténé. Cette approche est **insensible aux rebasages** : seul un changement de logique fonctionnelle modifie le hash.

**Listing 2: GhidraFunctionHasher.py (extrait)**

```python
# @category: Vulnerability Analysis
import hashlib, json

def get_opcode_hash(func):
    opcodes = ""
    listing = currentProgram.getListing()
    for cu in listing.getCodeUnits(
              func.getBody(), True):
        if isinstance(cu, Instruction):
            opcodes += cu.getMnemonicString()
    return hashlib.sha256(
               opcodes.encode()).hexdigest()

def run_analysis():
    results = []
    fm = currentProgram.getFunctionManager()
    for func in fm.getFunctions(True):
        results.append({
          "name":        func.getName(),
          "address":     str(func.getEntryPoint()),
          "size":        func.getBody()
                            .getNumAddresses(),
          "opcode_hash": get_opcode_hash(func),
          "calls": [f.getName() for f in
                func.getCalledFunctions(monitor)]
        })
```

## 4.2 Résultats d'extraction — libuclient.so

| Métrique | v33 | v35 |
|---|---|---|
| Fonctions extraites | 81 | 81 |
| Avec opcodes | 78 | 78 |
| Stubs PLT | 3 | 3 |
| Taille max (bytes) | 422 | 422 |
| Taille moyenne (bytes) | 49 | 49 |

**Table 5: Statistiques Ghidra — libuclient.so**

**Stubs PLT non désassemblés :** `__deregister_frame_info`, `__cxa_finalize` et `__register_frame_info` retournent `"no_instructions_found"` — entrées PLT non résolues statiquement.

## 4.3 Bug identifié — doublons de projet Ghidra

Les deux fichiers JSON exportés produisent **zéro différence** malgré des SHA-256 distincts sur les binaires sources.

**Listing 3: Diagnostic programmatique du bug**

```python
v33 = {f['name']:f for f in
       json.load(open('v33.json'))}
v35 = {f['name']:f for f in
       json.load(open('v35.json'))}
same = all(
  v33[n]['opcode_hash']==v35[n]['opcode_hash']
  for n in v33)
# => True  (artefact -- pas un resultat reel)
```

**Cause racine :** le script a été exécuté deux fois sur le *même programme chargé*. Ghidra maintient un cache par projet — si le fichier source n'est pas rechargé, l'analyse porte sur le même binaire.

**Correction :** créer un `New Project` Ghidra distinct pour chaque version, importer le fichier correspondant, analyser, puis exécuter le script.

**Impact :** les comparaisons d'opcodes restent à confirmer. Les résultats CVE (diff `sshkeys.js`, SHA-256 des binaires) ne sont pas affectés.

# 5 Phase 3 — Moteur de Diffing

## 5.1 Algorithme multi-critères

Le diff engine applique une comparaison hiérarchique :

(1) **Hash opcodes (SHA-256)** — détecte tout changement de logique assembleur
(2) **Taille en bytes** — détecte les ajouts même en cas de collision de hash
(3) **Similarité Jaccard** — mesure fine pour les cas ambigus

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}, \qquad \text{seuil} : J < 0.8 \tag{1}$$

**Listing 4: DiffEngine.py — comparaison multi-critères**

```python
def compare(v33_path, v35_path):
    v33 = {i['name']:i for i in
           json.load(open(v33_path))}
    v35 = {i['name']:i for i in
           json.load(open(v35_path))}
    modified, removed, added = [], [], []
    added = [n for n in v35 if n not in v33]
    for name, d33 in v33.items():
        if name in v35:
            d35 = v35[name]
            if (d33['opcode_hash']
                    != d35['opcode_hash']
                or d33['size']!=d35['size']):
                modified.append((name,d33,d35))
        else:
            removed.append(name)
    return added, removed, modified
```

## 5.2 Non-déterminisme de l'API Ghidra

`getCalledFunctions()` ne garantit pas un ordre stable, induisant des faux positifs sur la liste `calls` (8 fonctions affectées). Correction :

**Listing 5: Comparaison insensible à l'ordre**

```python
# Avant (faux positifs possibles)
if data_v33['calls'] != data_v35['calls']:...

# Apres (correction)
if set(data_v33['calls']) != \
   set(data_v35['calls']):...
```

# 6 Phase 4 — Corrélation CVE

## 6.1 CVE identifiés — v22.03.3 → v22.03.5

| CVE | CVSS | Description |
|---|---|---|
| CVE-2023-0464 | 7.5 | DoS OpenSSL — X.509 Policy Constraints |
| CVE-2023-0465 | 5.3 | Leaf cert policy silently ignored |
| CVE-2023-24181 | 5.4 | XSS — `/openvpn/pageswitch.htm` |
| CVE-2023-24182 | 5.4 | XSS stocké — `/system/sshkeys.js` |

**Table 6: CVE corrélés à ce patch OpenWRT**

## 6.2 CVE-2023-24182 — Confirmation

L'analyse diff de `sshkeys.js` révèle une modification chirurgicale sur la ligne 30 de l'interface de gestion des clés SSH de LuCI :

**Listing 6: Patch CVE-2023-24182 dans sshkeys.js**

```diff
# v22.03.3 -- vulnerable (string directe => XSS)
- E('pre', delkey)

# v22.03.5 -- patche (tableau => texte brut)
+ E('pre', [delkey])
```

**Mécanisme :** `delkey` était passée directement au DOM LuCI. Si la clé contenait du HTML/JavaScript, le navigateur l'interprétait (XSS stocké). Payload d'exemple :

```
ssh-rsa AAAA... <img src=x onerror=alert(1)>
```

L'enveloppement dans un tableau force LuCI à traiter la valeur comme texte brut, neutralisant l'injection HTML avec un seul caractère de différence.

**CVE-2023-24182 : CONFIRMÉ** — Patch localisé ligne 30, mécanisme compris. `E('pre', delkey)` → `E('pre', [delkey])`.

**CVE-2023-24181 :** Fichier `pageswitch.htm` absent — module OpenVPN non installé sur ce modèle TP-Link WDR4300. CVE non applicable.

## 6.3 CVE-2023-0464/0465 — En attente

Ces CVE affectent OpenSSL embarqué dans `libc.so`. Les SHA-256 distincts de `libuclient.so` constituent une présomption forte, mais la corrélation au niveau fonction nécessite les exports Ghidra corrects (cf. §4.3). Les fonctions candidates sont `X509_policy_check()`, `d2i_X509_POLICY_NODE()`, et les routines de validation de chaînes X.509.

# 7 Résultats & Analyse

| Élément analysé | Résultat | Statut |
|---|---|---|
| Extraction Binwalk v33/v35 | Succès complet | [OK] |
| Hash SHA-256 binaires ELF | 6 modifiés | [OK] |
| CVE-2023-24182 (XSS LuCI) | Confirmé, l.30 | [OK] |
| CVE-2023-24181 (XSS OpenVPN) | Module absent | [N/A] |
| Export Ghidra libuclient.so | Bug procédure | [KO] |
| Diff engine opcodes | 0 diff (bug) | [KO] |
| CVE-2023-0464/0465 OpenSSL | En cours | [WIP] |

**Table 7: Tableau de bord — état du projet**

La précision cible de 75 % sur la corrélation CVE/fonction est atteignable une fois les exports Ghidra corrects produits, au vu de la qualité de l'extraction (81 fonctions, hashes stables, `calls` répertoriés).

# 8 Conclusion

Ce projet a mis en place un pipeline complet de patch diffing sur firmwares IoT, depuis l'extraction Binwalk jusqu'à la corrélation CVE.

Le **CVE-2023-24182** a été confirmé et analysé : une modification d'une seule ligne dans `sshkeys.js` constitue le patch complet d'un XSS stocké critique dans LuCI. L'outillage Ghidra est opérationnel et extensible à tout binaire ELF MIPS/ARM. Un bug de procédure a été identifié programmatiquement — les exports corrects restent à produire pour finaliser la corrélation des CVE OpenSSL.

**Limites :** (1) stubs PLT non désassemblés en mode statique ; (2) similarité Jaccard peut produire des faux négatifs sur mnémoniques seuls ; (3) outil actuellement mono-binaire.

**Améliorations futures :** intégrer capstone pour un hashing au niveau des opérandes ; mode headless Ghidra ; rapport HTML interactif avec liens NVD.

**Bilan :** Malgré le bug d'export Ghidra, le pipeline global est validé. Le CVE-2023-24182 a été localisé et analysé avec précision. La correction de la procédure Ghidra permettra d'atteindre l'objectif de 75 % de précision.

## References


[1] NIST. *National Vulnerability Database.* https://nvd.nist.gov

[2] OpenWRT Project. *Release Notes 22.03.x.* https://openwrt.org

[3] NSA Cybersecurity. *Ghidra Software Reverse Engineering Framework.* https://ghidra-sre.org

[4] ReFirmLabs. *Binwalk Firmware Analysis Tool.* https://github.com/ReFirmLabs/binwalk

[5] Amadou Tidiane Anne. *Analyse de la pertinence des métriques système natives pour la détection d'anomalies sous Linux en environnements contraints.* HAL Open Science, 2026. https://hal.science/hal-05486729v1

[6] Google Project Zero. *0-day Vulnerabilities In The Wild.* https://googleprojectzero.github.io/0days-in-the-wild

[7] Quarkslab. *Comparative Analysis of Binary Diffing Tools.* Technical Blog, 2023.